

TASK 4: SYSTEM MODELLING AND DESIGN

*CEF440: INTERNET PROGRAMMING AND MOBILE
PROGRAMMING*

UNIVERSITY OF BUEA

FACULTY OF ENGINEERING AND
TECHNOLOGY

DEPARTMENT OF COMPUTER
ENGINEERING



REPUBLIC OF CAMEROON

PEACE-WORK-FATHERLAND

PAIX-TRAVAIL-PATRIE

Course Instructor : Dr Valery NKEMENI

SN	Names	Matricule	Speciality
1	Ayuk Nestor	FE22A164	Software
2	Bengha Paul	FE22A147	Network
3	Ngong Kong Steffy	FE22A262	Network
4	Njuma Joseph	FE22A247	Software
5	Oben Meg	FE22A288	Network

1. Introduction (to System Design and Modeling)

2. Overview of System Modeling Diagrams Utilized

2.1 Context Diagram

2.2 Dataflow Diagram (DFD)

2.3 Use Case Diagram

2.4 Sequence Diagram

2.5 Class Diagram

2.6 Deployment Diagram

3. Context Diagram for Mobile QoE Data Collection System

3.1 Introduction (to the specific Context Diagram)

3.2 Context Overview Diagram Description

3.3 External Entities and Data Flows

3.4 Definitions & Abbreviations

3.5 Conclusion

3.6 Figure

Image Reference

4. Data Flow Diagram of the Mobile QoE Data Collection App

4.1 Introduction to Data Flow Diagram (DFD)

4.2 Overview of the Level 1 DFD

4.3 Main Processes in the System (Level 1 DFD)

4.4 External Entities (Interactions with Level 1 DFD Processes) Backend Server

4.5 Conclusion (for DFD)

5. Use Case Diagram: Mobile QoE Data Collection System

5.1 Introduction

5.2 Actors

5.3 Use Cases

6. Sequence Diagrams – Advanced Scenarios

6.1 Introduction

6.2 Scenario 1: User Registration

6.2.1 Purpose 6.2.2 Objects/Lifelines

6.2.3 Step-wise Sequence

6.2.4 Postconditions

6.2.5 Alternate Flows

6.3 Scenario 2: User Login

6.3.1 Purpose

6.3.2 Objects/Lifelines

6.3.3 Precise Sequence

6.3.4 Postconditions

6.3.5 Alternate Flows

6.4 Scenario 3: User Onboarding (Initial App Setup post Login/Registration) (*Image Reference:*)

6.4.1 Purpose

6.4.2 Objects/Lifelines

6.4.3 Detailed Sequence

6.4.4 Postconditions

6.4.5 Alternate Flows

6.5 Scenario 4: User Manages Privacy Settings (*Image Reference:*)

6.5.1 Purpose

6.5.2 Objects/Lifelines

6.5.3 Detailed Sequence

6.5.4 Postconditions

6.5.5 Alternate Flows

6.6 Scenario 5: Prompted Feedback with Reward Trigger

6.6.1 Purpose

6.6.2 Objects/Lifelines

6.6.3 Step-by-step Description

6.6.4 Postconditions

6.6.5 Alternate Flows

6.7 Scenario 6: Automated Background Data Collection (Detailed Metrics) and Batched Upload

6.7.1 Purpose

6.7.2 Objects/Lifelines

6.7.3 Detailed Sequence

6.7.4 Postconditions

6.7.5 Alternate Flows

6.8 Scenario 7: MNO Accesses Aggregated QoE Data (via Dashboard/Portal) (*Image Reference:*)

6.8.1 Purpose

6.8.2 Objects/Lifelines

6.8.3 Detailed Sequence

6.8.4 Postconditions

6.8.5 Alternate Flows

7. User Experience Feedback System (Conceptual Class Structure/Modules)

7.1 Overview

7.2 User and Registration Modules

7.3 Experience Data Collection Module

7.4 Satisfaction Prompt Module

7.5 Feedback Logging Module

7.5.1 FeedbackLog Class

7.6 Inter-Class Relationships and System Flow

7.7 Overall Design Principles

7.8 Conclusion

8. Deployment Diagram Explanation: Mobile Network Quality of Experience (QoE) App

8.1 Introduction to Deployment Diagrams

8.2 High-Level Architecture

8.3 Node-by-Node Breakdown

8.4 Communication and Security

8.5 Real-World Analogy

8.6 Conclusion

9. Overall Conclusion of System Design and Modeling

1.0 Introduction

System design and modeling is an important software engineering phase where the behavior and system architecture are conceived, represented, and planned ahead of implementation. This phase makes use of visual models—like use case diagrams, sequence diagrams, and design patterns—to abstract and convey the composition, components, interaction, and data flow of a system. Clarity, user requirement alignment, and proper system decomposability to enable scalability, maintainability, and interoperability are aimed at. Conversion of user requirement and functional requirements to formalized models creates a basis for making knowledge-driven decisions and risk reduction across the software development life cycle.

2.0 Overview

This document provides a thorough collection of system modeling diagrams utilized to reflect the proposed solution from various viewpoints, including:

Context diagram: Describes the boundary of the system and its interfaces with external entities, providing an overall look at inputs and outputs.

Dataflow Diagram (DFD): Depicts the flow of information in the system, illustrating how the information flows between processes, data stores, and outside entities.

Use Case Diagram: Identifies functional requirements by describing the interaction between the users (actors) and the system in the form of defined use cases.

Sequence Diagram: Depicts the temporal sequence of interactions among system components or actors for specific scenarios, emphasizing message flow and execution order.

Class Diagram: Specifies the static system structure by modeling classes, their attributes and operations, and the relationships between them.

Deployment Diagram: Depicts the physical structure by illustrating how software components are allocated to physical nodes.

Collectively, the diagrams deliver a systematic and layered comprehension of the scheme, ranging from user interaction to information processing and physical deployment, to enable both technical implementation and communication to stakeholders.

3.0 Context Diagram for Mobile QoE Data Collection System

3.1. Introduction

The "Mobile QoE Data Collection System" represents an end-to-end platform to collect, process, analyze, and report mobile network users' Quality of Experience (QoE). This system collects both objective performance metrics and subjective user opinions, delivering valuable insights to Mobile Network Operators (MNOs) to further enrich the overall mobile service experience in markets such as Cameroon.

3.2. Context Overview Diagram

The context diagram graphically illustrates the interaction among the Mobile QoE Data Collection System (central process) and the external entities. There are specific data flows exchanged between each external entity and the central system. This top-down approach to representation provides an effective indication of system boundaries and interaction among stakeholders.

3.3. External Entities and Data Flows

A. Mobile Network Subscriber (User):

- **Data flows to the System:**
 - Registration Information (optional)
 - Login credentials (optional)
 - Subjective Comments and Ratings
 - Permission Grants (such as location access)
 - Privacy Preferences
 - Test Triggers (if implemented)
- **Data Flows from System:**
 - App Interface & Content
 - Feedback Prompts
 - Reward Notifications (if applicable)
 - Privacy Policy & Terms
 - System Status/Errors

B. Mobile Network Operator (MNO):

- **Data Transfer to the System:**
 - Configuration Parameters (KPI thresholds and alert settings)

- Reward System Integration APIs

- **Data Is Received by System:**

- Aggregated QoE Reports (visualizations, dashboards)
- Real-Time Critical Alerts
- Anonymous Data API Access
- Reward Confirmations (should it be incorporated)

C. Backend Administrator / App Development Team

- **Data Is Received by System:**

- System Configuration
- Software Updates & Admin Commands

- **System to Data Streams:**

- System health monitoring information
- Usage analytics
- Auditor Logs
- Raw & Processed Data Access

D. Reward Fulfillment System (Optional):

- **Data Flow to System:**

- Reward Status Updates

- **Data flows from the System:**

- Reward Issuance Requests

E. Device Operating System (iOS/Android)

- **Data To SEND To System:**

- Location, Timestamp, Network Status, Signal Strength, etc.
- Raw Network Metrics such as:
 - Upload Speed
 - Download Speed
 - Latency
 - Jitter

- Packet Loss
 - Bandwidth
- Device Information
- Background Task Triggers
- **Data Flows from System:**
 - Permission Requests
 - Notification Requests
 - Background task scheduling

3.4. Definitions & Abbreviations

- **QoE (Quality of Experience):** It indicates a user's overall satisfaction with a mobile network service.
- **Key Performance Indicator (KPI):** Numerical indicators used to measure the performance of the network.
- **MNO (Mobile Network Operator):** A mobile telecommunications company offering mobile communication services.
- **API (Application Programming Interface):** A collection of rules enabling systems to communicate.
- **XORs:** Stands for Exclusive ORs, which signifies optional paths where a single path is utilized at a time. Here, it just signifies optional paths for exchange of data such as login or registration.
- **RSSI, RSRP, RSRQ:** Parameters utilized to measure mobile network performance in terms of signal strength and quality.
- **Anonymous/Pseudonymized:** Information is treated to avoid identification of the person, following privacy regulations.
- **Latency:** The delay time in a system measured in milliseconds.
- **Jitter:** Packet arrival time variability, impacting real-time services such as VoIP.
- **Packet Loss:** Percentage of packets lost during transmission.
- **Upload/Download Speed:** The speed at which information is transmitted to, or received from, the internet, usually measured in Mbps.
- **Bandwidth:** The greatest rate at which a network connection transfers data.

3.5. Conclusion

Mobile QoE Data Collection System is a powerful multi-stakeholder platform allowing for effective feedback collection and performance measurement. It bridges the divide between users, MNOs, and backends and creates a self-reinforcing spiral of ongoing service improvement.

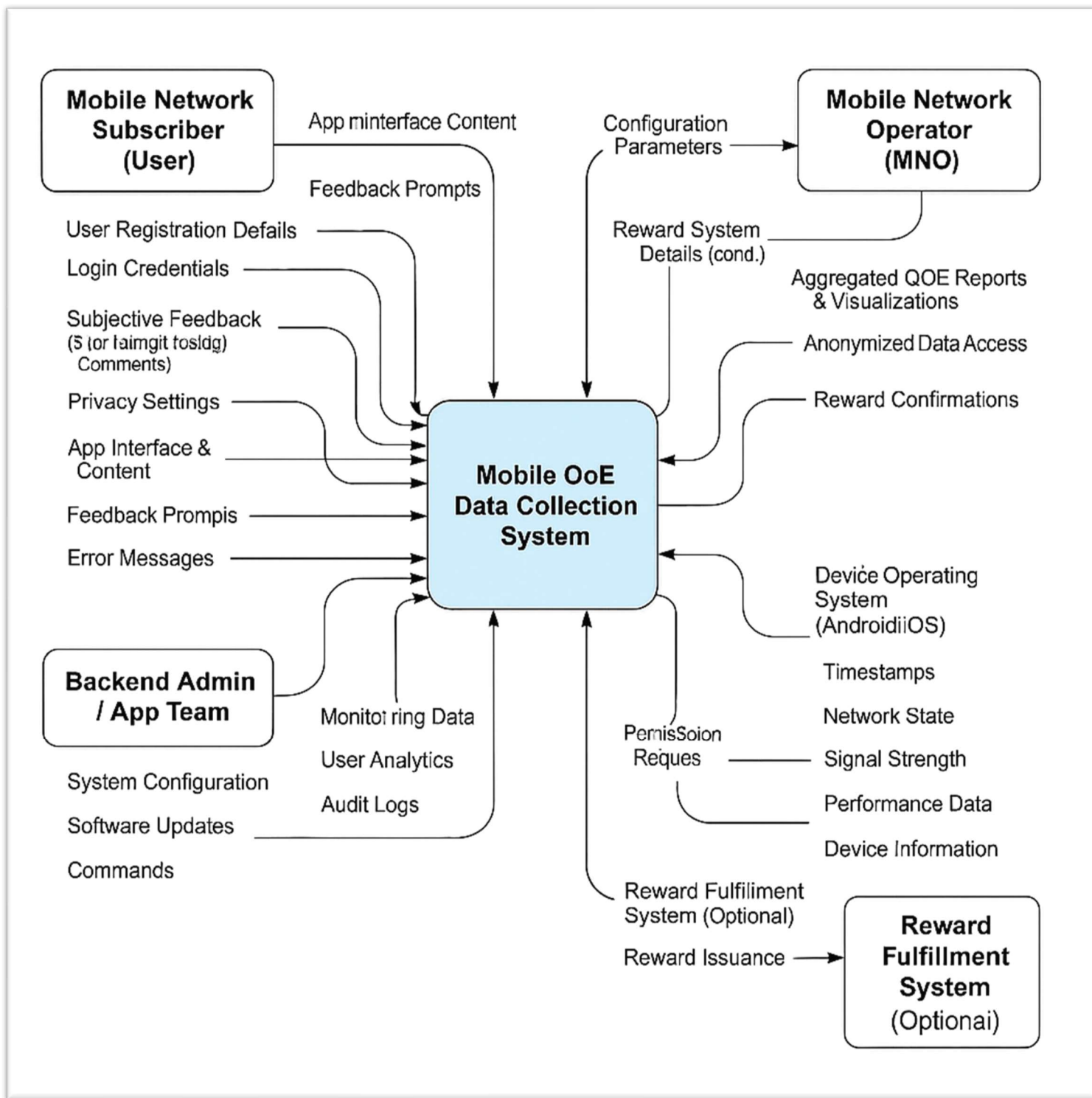


Fig 1.0 depicts the context diagram

4.0 Data Flow Diagram of the Mobile QoE Data Collection App

4.1. Introduction to Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) is a graph used to represent the flow of data throughout a system. It represents the processes responsible for converting data, the data stores, and the entities outside the system interacting with the system. DFDs are crucial during systems analysis and design to comprehend system behavior and the flow of data.

DFDs are usually organized in levels:

- **Level 0 (Context Diagram):** A top-level overview illustrating the entire system as a single process.
- **Level 1:** More detailed breakdown highlighting inner processes and information interaction.

4.2. Overview of the Level 1 DFD

The Level 1 DFD constructed for the Mobile QoE Data Collection App illustrates the inner operations of the mobile app built to collect both user opinions as well as objective network performance metrics. The app enables Mobile Network Operators (MNOs) to gain an understanding of the real user experiences and drive better service delivery.

4.3. Main Processes in the System

1. Gather Background Information (1.0)

- Collects network type, signal strength, location, and timestamp from the device's operating system.

2. Prompt & Collect Feedback (2.0)

- Alerts user based on issues detected; gathers comments/ratings.

3. Save Data Locally (3.0)

- Stores collected data (objective + subjective) in local storage when offline.

4. Upload Data to Backend (4.0)

- Sends stored information to the backend server securely using the API (real-time or batch mode).

5. Privacy & Permissions Management (5.0)

- Manages user consent, permissions (location, notifications), and opt-out of data.

4.4. External Entities

- **Mobile Subscribers**
 - Provide control over privacy settings (interact with 2.0 and 5.0).
- **Mobile Network Operators**
 - Provide location and network information (interact with 1.0).

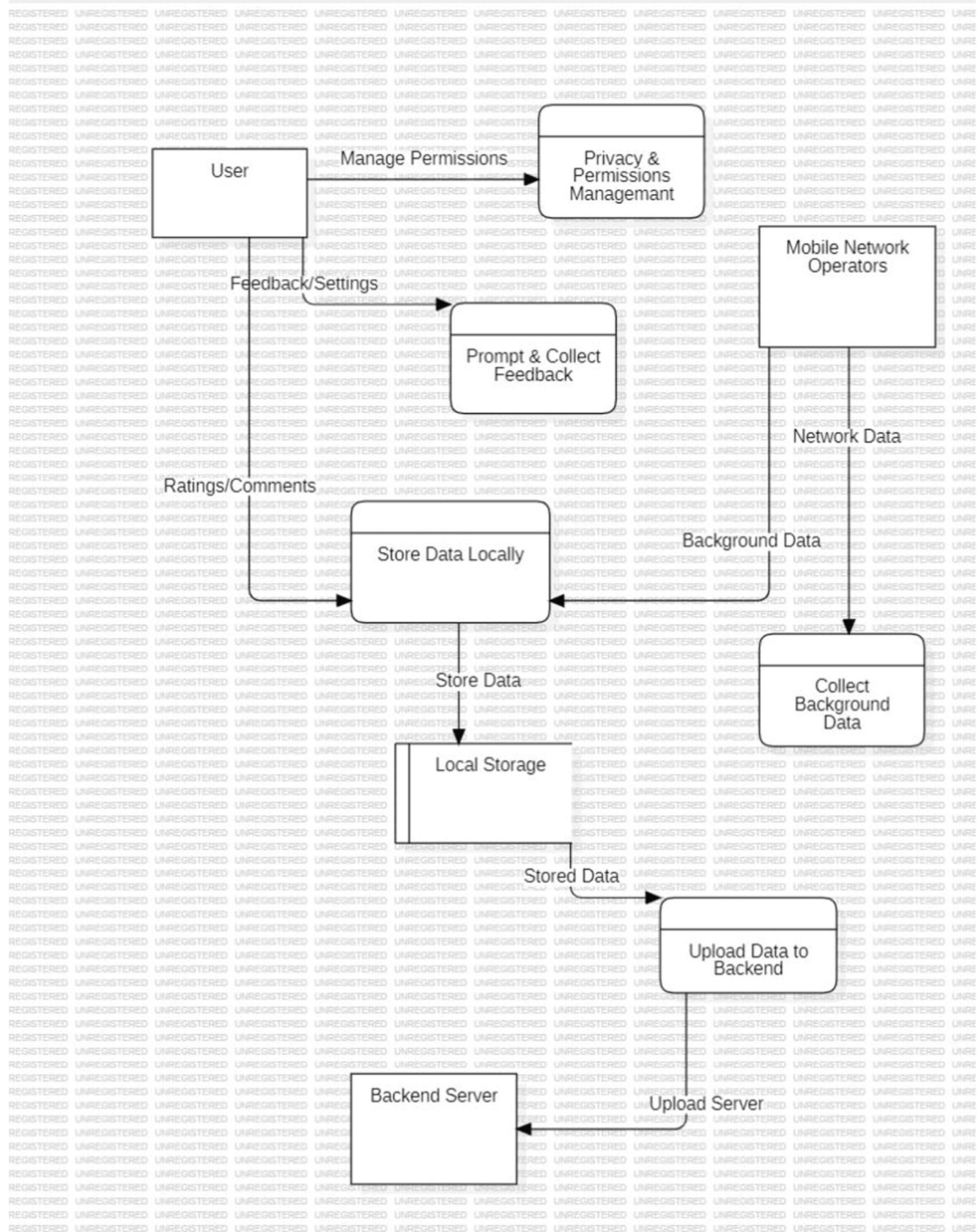


Fig 2.0 depicts the data flow diagram

4.5. Conclusion

The DFD accurately depicts how the mobile app acquires, processes, saves, and communicates information, as well as protects user privacy. It provides an accurate blueprint for system design and conforms to the project's functional and non-functional requirements.

5.0 Use Case Diagram: Mobile QoE Data Collection System

This document describes the use case diagram for the Mobile QoE Data Collection System. The system involves various actors such as Mobile Network Subscribers (Users), Mobile Network Operators (MNO), App Development Team, and Reward Fulfillment System. The use case diagram illustrates the interactions between these actors and the system.

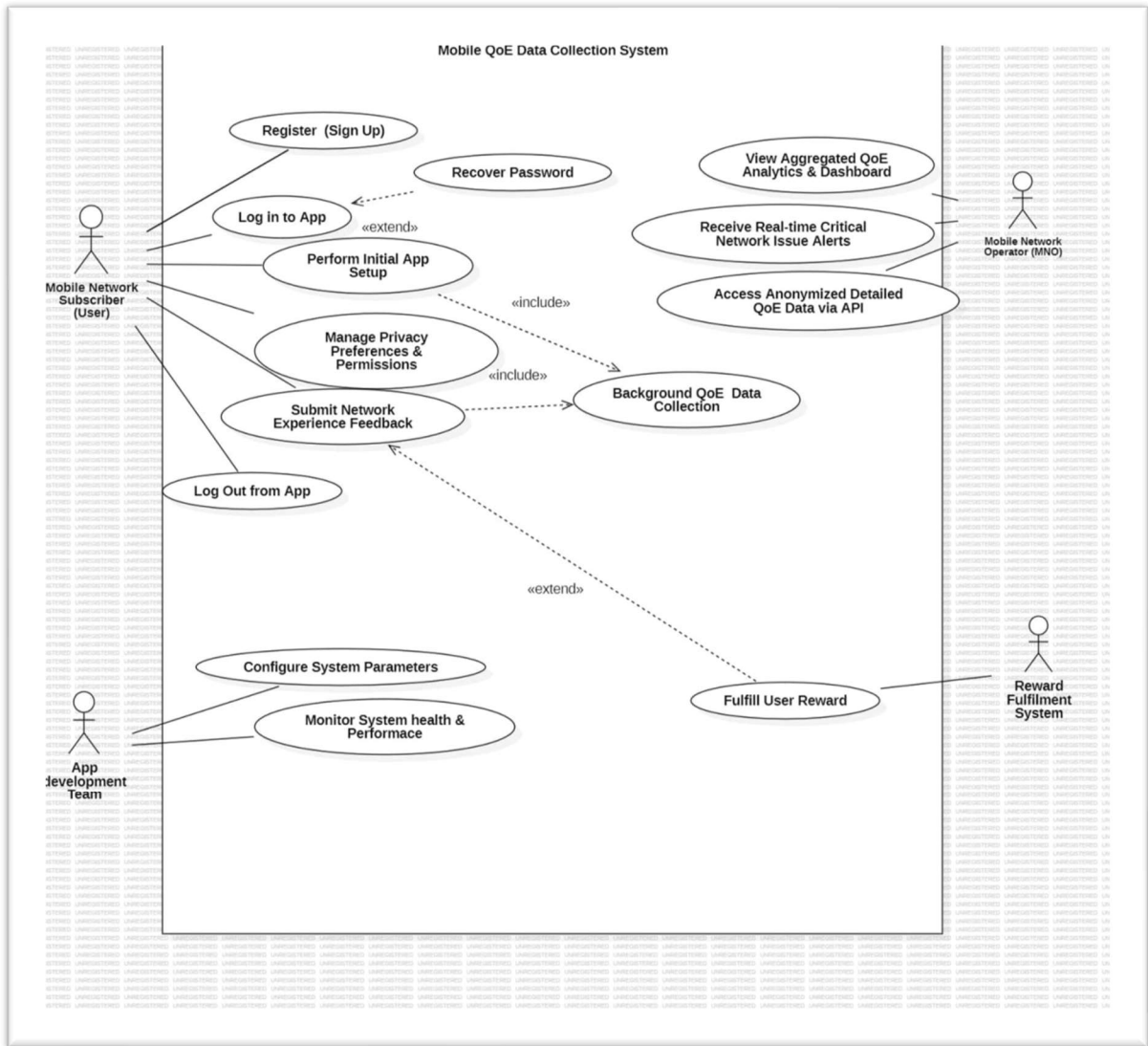


Fig 3.0 depicts the use case diagram

5.1 Actors

- Mobile Network Subscriber (User): Interacts with the app to register, log in, provide feedback, etc.
- Mobile Network Operator (MNO): Views aggregated data and receives critical alerts.
- App Development Team: Configures and monitors the system.

- Reward Fulfillment System: Manages user rewards.

5.2 Use Cases

- Register (Sign Up): User registers for the app.
- Recover Password: Allows user to recover account credentials.
- Log in to App: Authenticates user.
- Perform Initial App Setup: App initial configuration by the user.
- Manage Privacy Preferences & Permissions: User sets privacy controls.
- Submit Network Experience Feedback: User provides feedback on network quality.
- Background QoE Data Collection: Continuous data collection from user's device.
- Log Out from App: Ends user's session.
- Configure System Parameters: Admin-level configuration by dev team.
- Monitor System Health & Performance: Monitors operational status.
- View Aggregated QoE Analytics & Dashboard: Operator views overall quality metrics.
- Receive Real-time Critical Network Issue Alerts: Operator is notified of issues.
- Access Anonymized Detailed QoE Data via API: Operator accesses detailed data.
- Fulfill User Reward: Rewards are managed based on user participation.

6.0 Sequence Diagrams – Advanced Scenarios

6.1 Introduction

Sequence diagrams play a central role in system design since they represent the dynamic system behavior, showing how various objects and components cooperate with each other over time to carry out a series of tasks or use cases. Sequence diagrams represent messages between objects in a time-ordered manner, thus making the control and data flow clearer.

Sequence diagrams are crucial to the Mobile QoE Data Collection System for:

- Describing the sequential execution procedures of key activities.
- Specifying the roles of different software objects (both in the mobile application and the server side).
- Understanding the synchronous and asynchronous messages exchanged between these objects.
- Identification of potential interaction flow points of failure or bottlenecks.
- Giving a precise instructional guide to the developers at the implementation stage.

The below sections outline the in-depth sequence of key scenarios, covering user authentication (login, registration, password reset), user onboarding, core collection and submission processes for key data, privacy management, and MNO data access. These scenarios guarantee all major functional requirements as well as the treatment of key QoE metrics (e.g., Location, Timestamps, Network Type, Signal Strength, Upload/Download Speeds, Latency, Jitter, Packet Loss, and Bandwidth) are accurately mapped to object interaction.

Scenario 1: User Registration

Purpose:

To allow a new user to sign up for a secure account to utilize the application.

Objects/Lifelines:

User, RegistrationUI (App), AccountHandler (App), APIClient (App), UserAccountService (Backend), UserProfileDB (Backend), (Optional) EmailVerificationService (Backend)

Step-wise Sequence:

- The user inputs registration information into RegistrationUI(App) and clicks "Register".
- RegistrationUI calls AccountHandler.registerUser()
- AccountHandler performs client-side validation
 - If invalid, shows an error message
 - If valid: Calls APIClient.createUserAccount()
- APIClient makes a POST request to /users/register endpoint
- UserAccountService performs server-side validation

- If fails: Returns errorResponse
 - If successful: Hashes the password and saves user info to UserProfileDB
 - If applicable: Triggers EmailVerificationService
- Returns registrationSuccessResponse to APIClient
- APIClient sends response to AccountHandler
- AccountHandler refreshes RegistrationUI
- RegistrationUI displays a success message

Postconditions:

User account successfully created. Email might be pending verification.

Alternate Flows:

Invalid input; username/email already taken; network/server/email errors

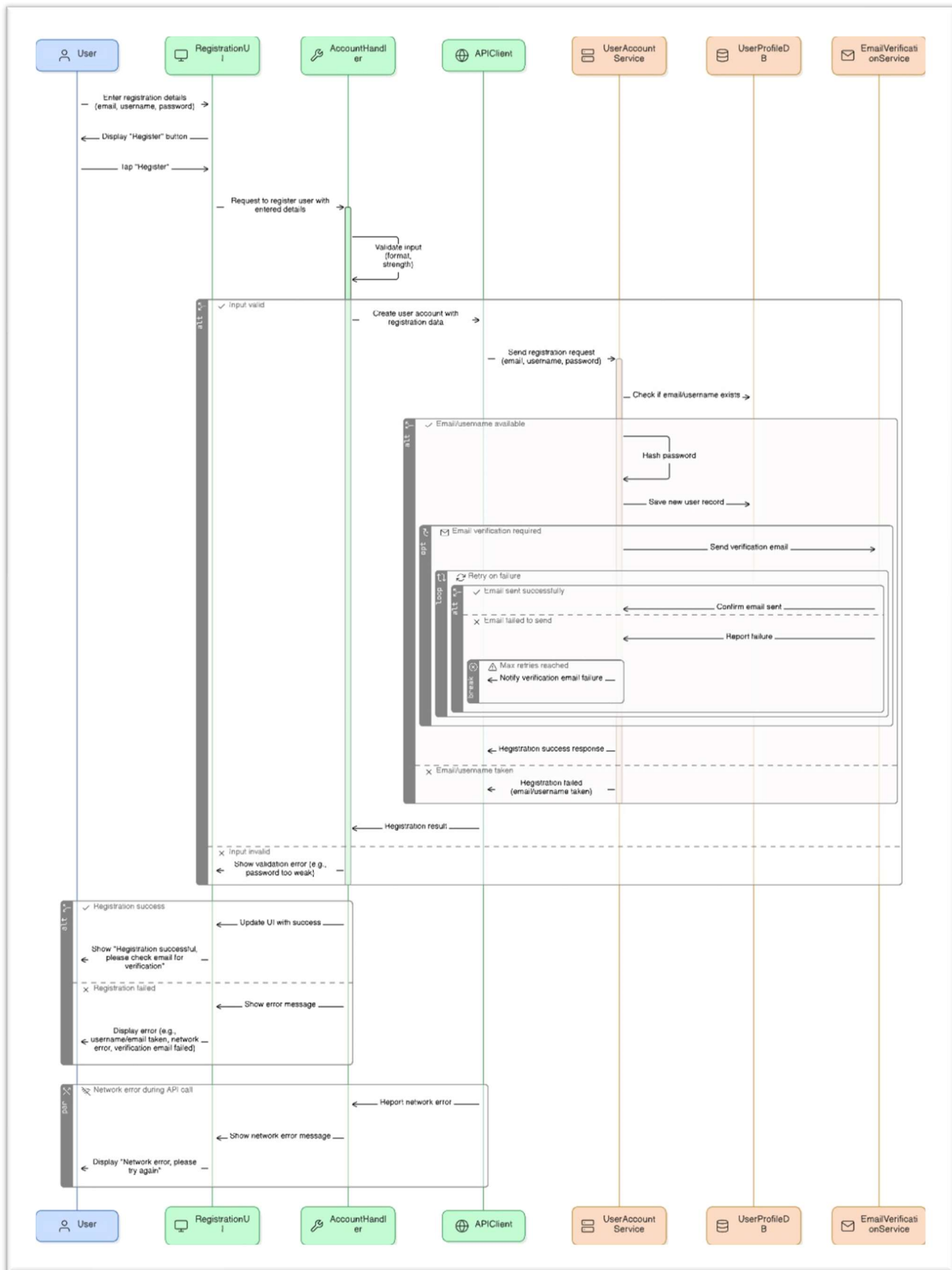


Fig 6.0 depicts the sequence diagram for user registration

Scenario 2: User Login

Purpose:

Enables a registered user to log in and authenticate.

Objects/Lifelines:

User, LoginUI (App), AuthHandler (App), APIClient (App), AuthService (Backend), UserProfileDB (Backend), SessionManager (App)

Precise Sequence:

- User inputs credentials into LoginUI(App) and clicks "Login"
- LoginUI calls AuthHandler.loginUser()
- AuthHandler calls APIClient.authenticateUser()
- APIClient sends a POST request to /auth/login endpoint
- AuthService retrieves user info from UserProfileDB
 - If user not found or not verified: Returns authenticationFailureResponse
 - On password mismatch: Returns authenticationFailureResponse
 - If valid:
 - Creates sessionToken
 - Logs login event
 - Returns loginSuccessResponse
- APIClient transfers data to AuthHandler
- AuthHandler saves session via SessionManager and updates LoginUI
- LoginUI navigates the user to the main screen

Postconditions:

User is logged in; session is securely established

Alternate Flows:

Incorrect credentials; unverified account; server/network issues

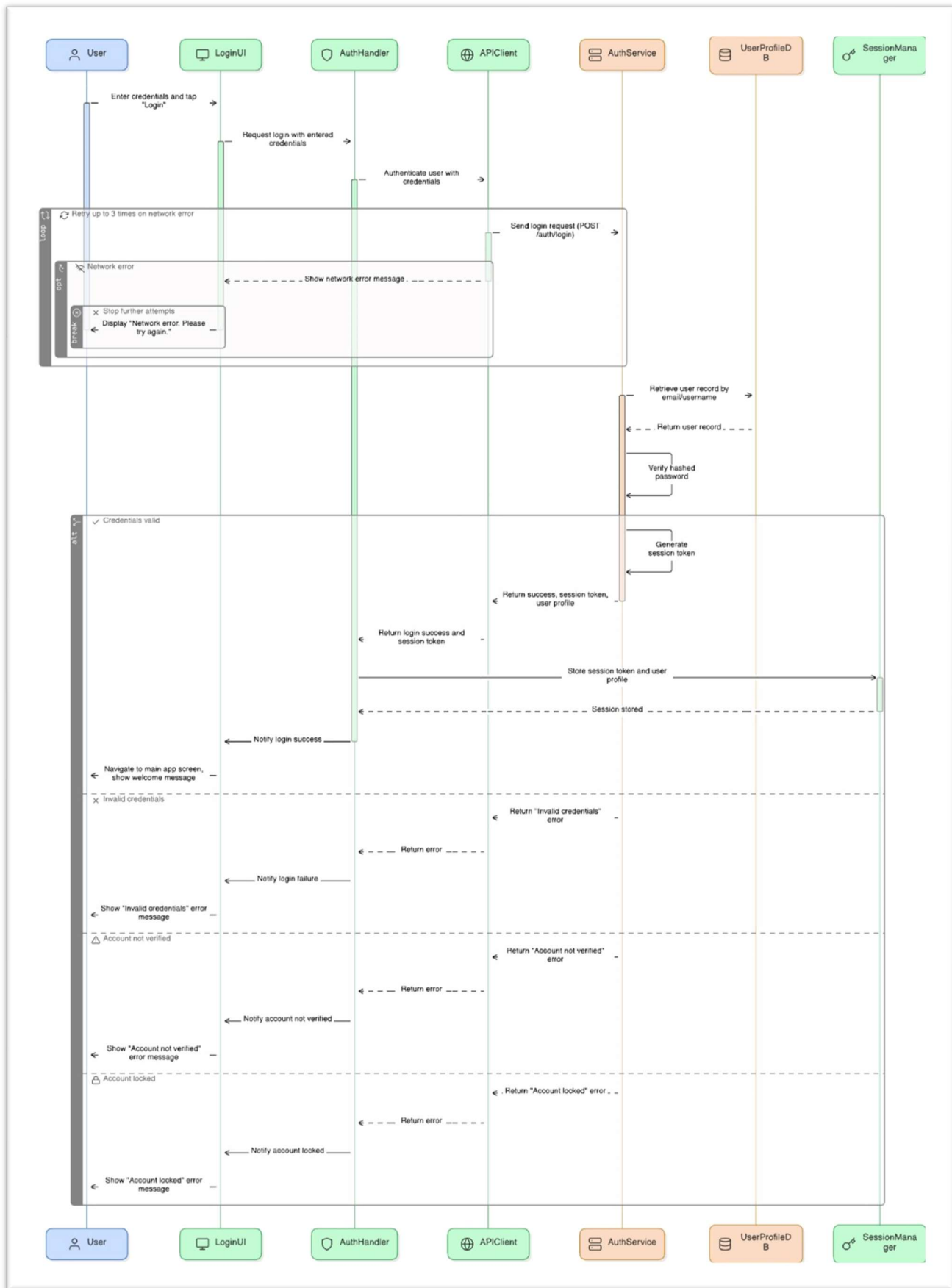


Fig 6.1 depicts the sequence diagram for user login

User Onboarding (Initial App Setup post Login/Registration)

Purpose: Details the sequence of interactions when a user sets up the app for the first time after successfully registering and logging in, focusing on permission grants and privacy consent.

- **Objects/Lifelines:** User, OnboardingUIManager(App), PermissionHandler(App), OSPermissionManager(DeviceOS), PrivacyConsentHandler(App), APIClient(App), UserProfileService(Backend), LocalStorage(App).
- **Detailed Sequence:**
 1. User: Has successfully logged in (this triggers onboarding if it's the first login or setup isn't complete).
 2. OnboardingUIManager(App): Displays a welcome message and a clear explanation of the app's purpose, detailing the types of data that will be collected (specifically **Location, Timestamps, Network Type, Signal Strength, Upload/Download Speeds, Latency, Jitter, Packet Loss, Bandwidth**) and how this data contributes to network improvement.
 3. User: Interacts with the UI to proceed to the next step.
 4. OnboardingUIManager(App): Displays the full Privacy Policy and Terms of Service for user review.
 5. OnboardingUIManager(App): Invokes PrivacyConsentHandler.requestConsent() to formally request user agreement.
 6. User: Interacts with the UI to explicitly accept the consent terms.
 7. PrivacyConsentHandler(App): Records consentStatus = true.
 8. PrivacyConsentHandler(App): Calls LocalStorage.saveConsentStatus(true) to persist consent locally.
 9. PrivacyConsentHandler(App): Calls APIClient.updateUserConsent(userID, consentStatus: true) to inform the UserProfileService(Backend).
 10. OnboardingUIManager(App): Iteratively, for each required OS permission (e.g., Precise Location, Background App Activity, Notifications, Network State Access):
 - Displays a clear justification explaining why the permissionType is needed for app functionality (e.g., "Location is used to correlate network quality with specific geographical areas").
 - Invokes PermissionHandler.requestPermission(permissionType).
 - PermissionHandler(App): Invokes OSPermissionManager.promptForPermission(permissionType).
 - OSPermissionManager(DeviceOS): Displays the OS-native permission prompt to the User.
 - User: Grants or denies the permissionType via the OS prompt.

- OSPermissionManager(DeviceOS): Returns permissionStatus (granted/denied) to PermissionHandler(App).
- PermissionHandler(App): Records permissionStatus locally. Informs OnboardingUIManager(App).
- PermissionHandler(App): Calls LocalStorage.savePermissionStatus(permissionType, permissionStatus).
- PermissionHandler(App): Calls APIClient.updateUserPermission(userID, permissionType, permissionStatus) to inform UserProfileService(Backend).
- **If** a critical permissionType is denied: OnboardingUIManager(App) displays a message to the User explaining the impact on app functionality (e.g., "Without location access, we cannot provide location-specific network quality data").

11. OnboardingUIManager(App): Displays "Setup Complete" message to the User.

12. OnboardingUIManager(App): Notifies relevant internal app services (e.g., BackgroundQoECollectorService) that onboarding is complete and current permission states are set, allowing them to commence operations if permitted.

- **Postconditions:** User permissions and privacy consent are recorded locally on the device and synchronized with the backend. Background data collection services can start if all necessary permissions have been granted.
- **Alternate Flows:**
 - User denies consent to Privacy Policy: Onboarding cannot complete; app functionality remains locked or exits.
 - User denies a non-critical permission: App proceeds with limited functionality, clearly communicating these limitations to the user.
 - User quits onboarding mid-way: The app saves the current state of consent and granted permissions, allowing the user to resume onboarding upon next app launch.

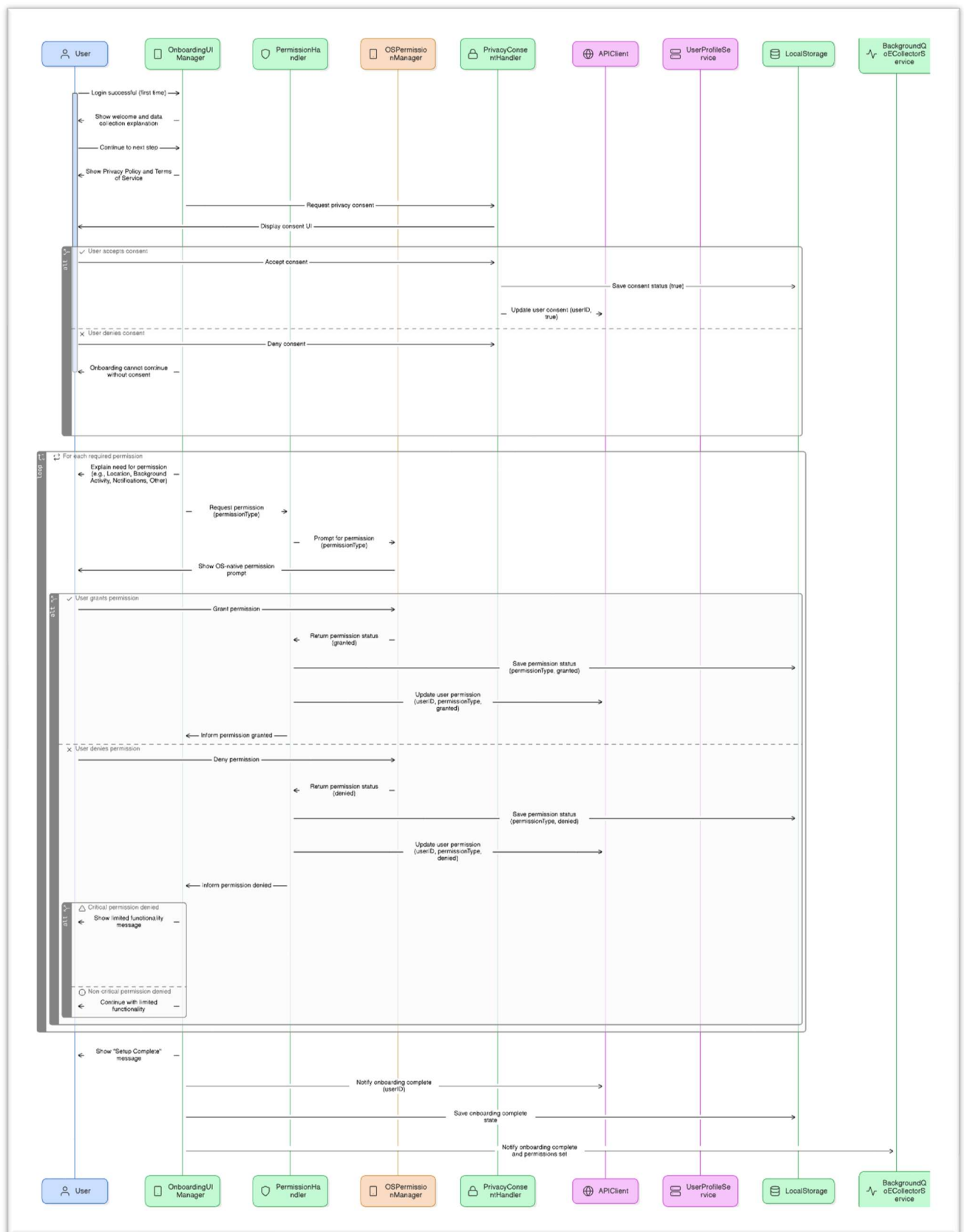


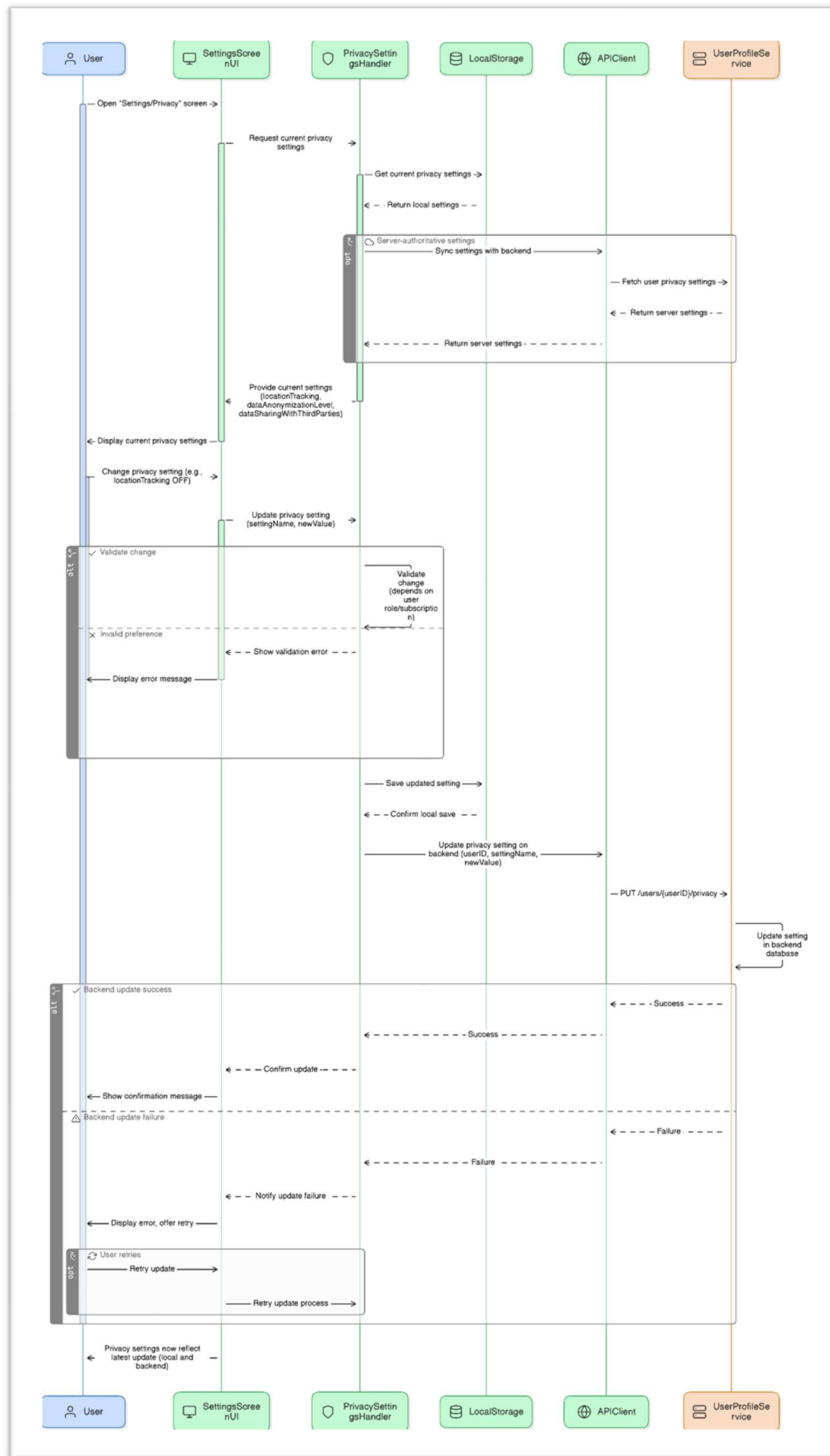
Fig 6.2 depicts the sequence diagram for user on-boarding

Scenario 5: User Manages Privacy Settings

Purpose: Details how a user changes their privacy preferences (e.g., location tracking, data sharing granularity) after the initial onboarding process.

- **Objects/Lifelines:** User, SettingsScreenUI(App), PrivacySettingsHandler(App), LocalStorage(App), APIClient(App), UserProfileService(Backend).
- **Detailed Sequence:**
 1. User: Navigates to the "Settings" or "Privacy" section within the SettingsScreenUI(App).
 2. SettingsScreenUI(App): Invokes PrivacySettingsHandler.getCurrentSettings() to fetch the user's current privacy preferences.
 3. PrivacySettingsHandler(App):
 - Retrieves current settings from LocalStorage(App).
 - (Optionally, if server is authoritative or for sync) Calls APIClient.fetchUserPrivacySettings(userID) which GETs data from /users/{userID}/privacy endpoint of UserProfileService(Backend). UserProfileService(Backend) returns current server-side settings. APIClient(App) returns these to PrivacySettingsHandler(App) which reconciles with local settings if needed.
 4. PrivacySettingsHandler(App): Returns currentSettings (e.g., {locationTrackingEnabled: true, dataAnonymizationLevel: 'enhanced', callLogAccessPermitted: false}) to SettingsScreenUI(App).
 5. SettingsScreenUI(App): Displays the currentSettings to the User using appropriate UI controls (e.g., toggles, dropdowns).
 6. User: Modifies a setting (e.g., toggles locationTrackingEnabled from true to false).
 7. SettingsScreenUI(App): Invokes PrivacySettingsHandler.updateSetting(settingName: 'locationTrackingEnabled', newValue: false).
 8. PrivacySettingsHandler(App): Validates the newValue for the settingName.
 - **If invalid:** Returns validationError to SettingsScreenUI(App), which displays an error message.
 9. PrivacySettingsHandler(App): If valid, calls LocalStorage.saveSetting(settingName, newValue) to persist the change locally.
 10. PrivacySettingsHandler(App): Calls APIClient.updateUserPrivacySetting(userID, settingName, newValue) to synchronize the change with the backend.

11. `APIClient(App)`: Sends a PUT or POST request to `/users/{userID}/privacy` endpoint of `UserProfileService(Backend)` with the `settingName` and `newValue`.
 12. `UserProfileService(Backend)`: Updates the setting in the backend user profile database and returns success or failure status.
 13. `APIClient(App)`: Returns the `backendUpdateStatus` to `PrivacySettingsHandler(App)`.
 14. `PrivacySettingsHandler(App)`:
 - **If `backendUpdateStatus` is failure:** May initiate a retry logic or mark the local setting as "pending sync". Informs `SettingsScreenUI(App)` of potential sync issue.
 - **If `backendUpdateStatus` is success:** Confirms update to `SettingsScreenUI(App)`.
 15. `SettingsScreenUI(App)`: Displays a confirmation message (e.g., "Settings updated successfully") to the User.
- **Postconditions:** The user's privacy setting is updated both locally on the device and on the backend server. Subsequent data collection and processing by the app adheres to the new setting.
 - **Alternate Flows:**
 - Backend update fails: The local change might be rolled back, or marked as pending synchronization, with the user being informed of the sync issue.
 - User attempts to set an invalid preference value (e.g., if validation fails at step 8).
 - Network connectivity issue prevents immediate sync with backend: Setting saved locally and queued for sync.



Prompted Feedback with Reward Trigger

Purpose:

Describes the interaction sequence when a user answers a feedback question, including the gathering of in-depth contextual metrics and the conditional release of a reward.

Objects/Lifelines:

User, MobileAppUI, FeedbackHandler (App), ContextualDataCollector (App: modules for Location, NetworkInfo, PerformanceMetrics like Speed, Latency, Jitter, etc.), LocalStorage (App), APIClient (App), BackendAPI, QoEDataProcessor (Backend), RewardService (Backend - conditional on MNO agreement), SessionManager (App)

Step-by-step Description:

- System recognizes event (e.g., call drop, bad speed) and notifies MobileAppUI through displayFeedbackPrompt()
- MobileAppUI displays notification or prompt
- User provides input to prompt and submits the feedback
- MobileAppUI obtains sessionToken
- If no valid session: redirects user to Login
- If session is valid: FeedbackHandler.processSubmission() is called
- FeedbackHandler calls ContextualDataCollector.getCurrentContext()
- Collector modules collect metrics: location, network type, signal strength, speeds, latency, etc.
- Full context is returned
- FeedbackHandler generates feedbackRecord and saves it locally
- UI confirms feedback saved
- If the reward system is enabled and MNO permits: APIClient.triggerReward() is invoked
- APIClient sends the token and feedback ID to the backend
- BackendAPI validates and routes to RewardService
- RewardService handles eligibility and returns status
- APIClient can trigger uploadQueuedData()

Postconditions:

Locally stored context and feedback; reward delivered if applicable; data queued.

Alternate Flows:

Prompt rejected; offline reward upload (in the queue); reward failure.

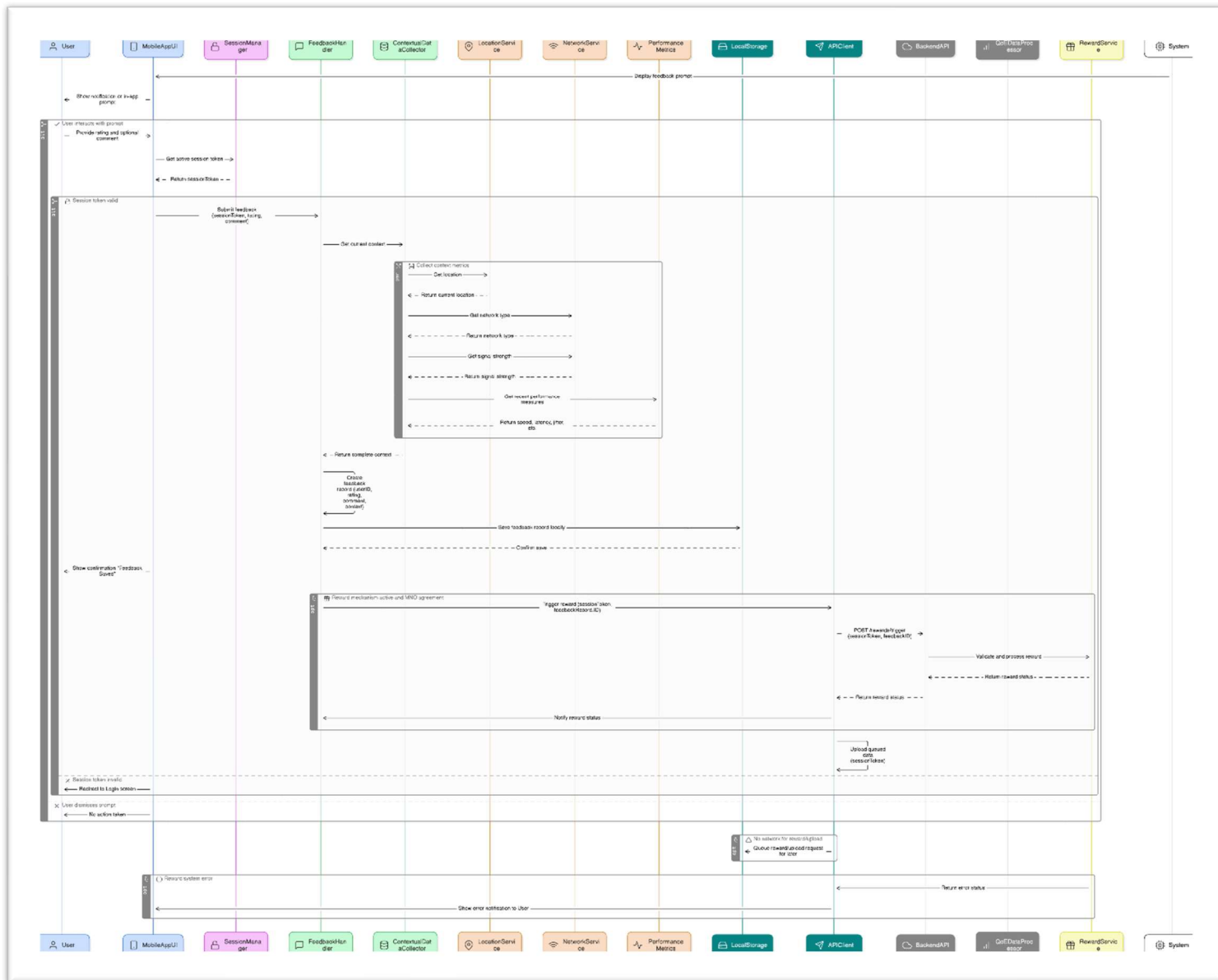


Fig 6.4 depicts the sequence diagram for prompted feedback

Automated Background Data Collection (Detailed Metrics) and Batched Upload

Purpose:

Defines automatic collection of QoE metrics at the backend and uploading in batches.

Objects/Lifelines:

OSTaskScheduler, BackgroundQoECollectorService (App), MetricSampler (App), LocalStorage (App), APIClient (App), BackendAPI, BatchDataIngestor (Backend), SessionManager (App)

Detailed Sequence:

- OSTaskScheduler invokes BackgroundQoECollectorService.executeTask()
- Service obtains session information from SessionManager.getActiveSessionInfo()
- If invalid session: might stop or collect anonymously
- If valid: MetricSampler.collectAllMetrics() is invoked
- Parameters measured: location, time, network type, signal strength, speeds, latency, jitter, packet loss, bandwidth
- metricRecord is stored locally and persisted
- Checks to see if upload is allowed via NetworkMonitor.isDataUploadAllowed()
- If permitted and batch condition is satisfied:
 - Retrieves pending records
 - Sends POST /data/batch_upload to BackendAPI
 - Backend validates and passes to BatchDataIngestor
 - Data ingested, result returned
 - Marks uploaded records via LocalStorage.markRecordsAsUploaded()
 - Execution completes; next run can be scheduled

Postconditions:

If conditions permit, metrics are collected and uploaded in batches.

Alternate Flows:

User logout; OS limits tasks; no network (data pending); batch upload fails (retry later).

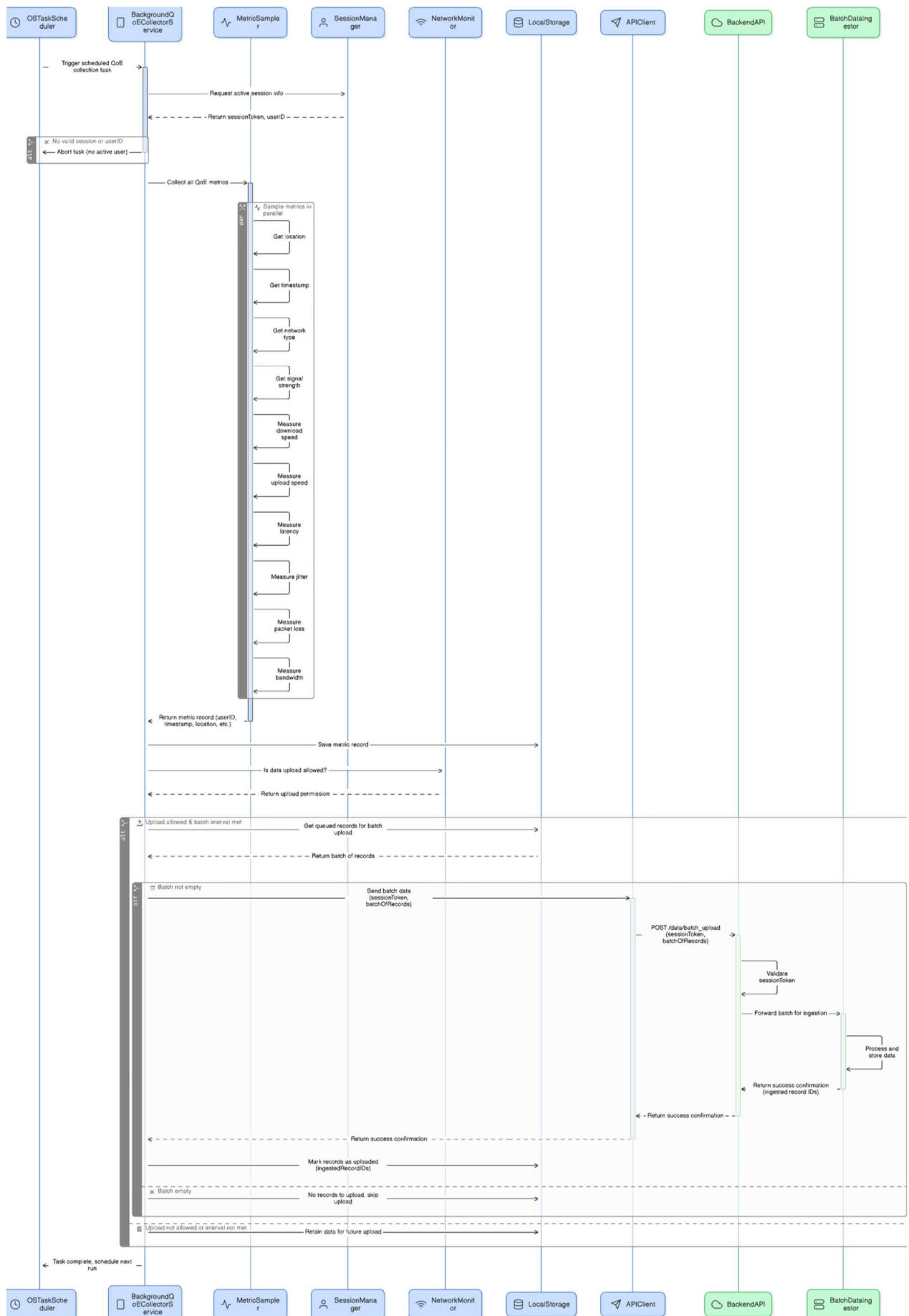
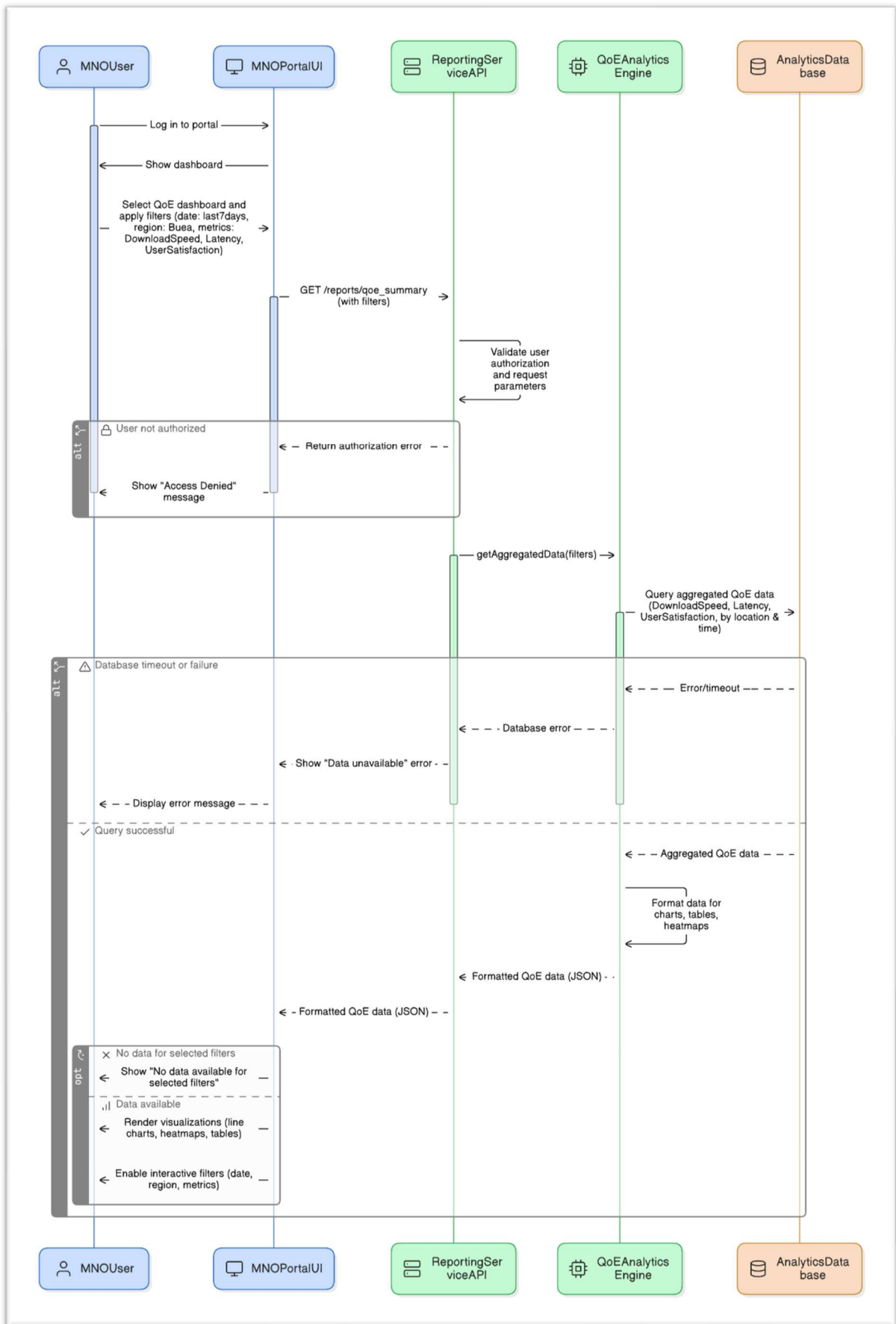


Fig 6.5 depicts the sequence diagram for background data collection

MNO Accesses Aggregated QoE Data (via Dashboard/Portal)

- **Purpose:** Illustrates how an MNO representative views summarized QoE data.
- **Objects/Lifelines:** MNOUser, MNOPortalUI(Web), ReportingServiceAPI(Backend), QoEAnalyticsEngine(Backend), AnalyticsDatabase(Backend).
- **Detailed Sequence:**
 1. MNOUser: Logs into MNOPortalUI.
 2. MNOUser: Navigates to QoE dashboard and applies filters (e.g., dateRange: 'last7days', region: 'Buea', metrics: ['DownloadSpeed', 'Latency', 'UserSatisfaction']).
 3. MNOPortalUI: Sends GET request to ReportingServiceAPI at /reports/qoe_summary with filter parameters.
 4. ReportingServiceAPI: Validates MNO user's authorization and request parameters.
 5. ReportingServiceAPI: Calls QoEAnalyticsEngine.getAggregatedData(filters).
 6. QoEAnalyticsEngine: Queries AnalyticsDatabase using the provided filters to retrieve aggregated data for **DownloadSpeed trends, Latency distributions, UserSatisfaction scores, etc., correlated with Location and Timestamps.**
 7. AnalyticsDatabase: Returns aggregated qoeData.
 8. QoEAnalyticsEngine: Formats qoeData (e.g., for charts, tables, heatmaps). Returns formatted data to ReportingServiceAPI.
 9. ReportingServiceAPI: Returns formatted data (e.g., JSON) to MNOPortalUI.
 10. MNOPortalUI: Renders the visualizations (charts, maps, tables) for MNOUser.
- **Postconditions:** MNO user views and analyzes QoE trends.
- **Alternate Flows:** No data available for selected filters; MNO user lacks authorization for certain data views; Database query timeout.



7.0 CLASS DIAGRAM

OVERVIEW

A comprehensive and well-structured system designed to capture, analyze, and utilize user experience and satisfaction data. It consists of multiple interconnected modules that operate seamlessly to ensure robust registration, interaction tracking, prompt feedback collection, and detailed feedback logging.

The architecture emphasizes **modularity**, **data integrity**, and **traceability**, allowing for continuous system improvement through the ongoing analysis of user interactions and feedback.

7.1. USER AND REGISTRATION MODULES

USER / ACCOUNT CLASS

- **Attributes:**
Includes unique identifiers (e.g., UserID), personal details such as Name and Email, security credentials like Password, and RegistrationDate.
- **Methods:**
 - register() – Handles new account creation.
 - updateProfile() – Allows modification of user information.
- **Role:**
Represents the core user model, encapsulating essential data for identification and profile management.

USER REGISTRATION MODULE

- **Attributes & Methods:**
Encapsulates the registration workflow with:
 - validateUserInput() – Ensures all user input meets criteria.
 - initiateRegistration() – Launches the registration process.
- **Role:**
By isolating registration logic, this module promotes modularity, ensures data integrity from the start, and simplifies maintenance and scalability.

7.2. EXPERIENCE DATA COLLECTION MODULE

EXPERIENCEDATA CLASS

- **Attributes:**
Records key interaction data such as ExperienceID (unique), Timestamp, Duration, and EventType.
- **Methods:**

- captureExperience() – Logs user activity in real time.
- analyzeMetrics() – Performs initial evaluation on collected data.
- **Role:**
Serves as the foundational component for tracking user engagement, building a comprehensive dataset to connect behavior with satisfaction indicators.

7.3. SATISFACTION PROMPT MODULE

SATISFACTIONPROMPT CLASS

- **Attributes:**
Includes PromptID, the actual PromptMessage, DisplayTime to manage timing, and multiple ResponseOptions.
- **Methods:**
 - displayPrompt() – Presents contextual questions to users.
 - recordResponse() – Immediately logs user ratings or responses.
- **Role:**
Facilitates timely feedback collection by prompting users when their input is most relevant.

7.4. FEEDBACK LOGGING MODULE

FEEDBACKLOG CLASS

- **Attributes:**
Contains FeedbackID, user Comments, numeric Rating, and a Timestamp.
- **Methods:**
 - storeFeedback() – Adds new entries.
 - updateFeedback() – Modifies existing feedback.
 - retrieveFeedback() – Accesses historical data.
- **Role:**
Acts as a central repository for feedback, supporting trend analysis, continuous improvement, and historical tracking.

7.5. INTER-CLASS RELATIONSHIPS AND SYSTEM FLOW

- **User ↔ ExperienceData:**
Each user is linked to multiple experience records, enabling accurate tracking across the user journey.
- **ExperienceData ↔ SatisfactionPrompt:**
User actions dynamically trigger relevant satisfaction prompts, ensuring sentiment is captured in real time.
- **SatisfactionPrompt ↔ FeedbackLog:**
Feedback from prompts is logged directly, creating a traceable path from interaction to stored insight.

Overall Design Principles

- **Modularity & Separation of Concerns:**

Each class focuses on a distinct responsibility—registration, experience capture, feedback prompting, or logging—enhancing scalability and maintainability.

- **Data Integrity & Traceability:**

Unique identifiers across modules ensure complete traceability, enabling reliable analysis and continuous enhancement of the user experience.

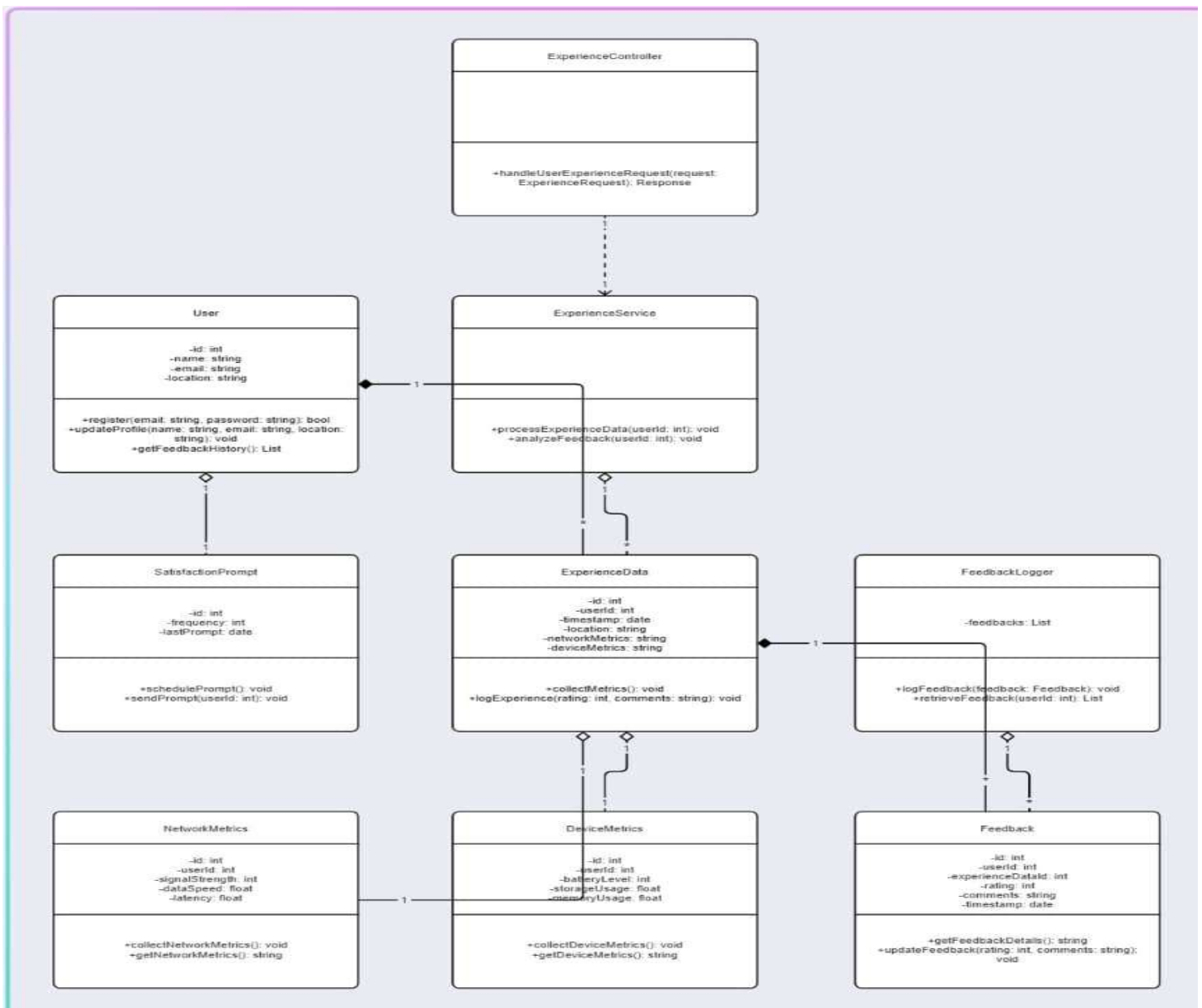


Fig 7.0 depicts interactions within the class diagram

CONCLUSION

This user-focused architecture connects registration, engagement tracking, satisfaction prompts, and feedback management into a cohesive system. Each module plays a distinct role while integrating smoothly across the user journey—from onboarding to continuous feedback.

The modular design promotes maintainability and scalability. Emphasis on data integrity and traceability enables organizations to extract actionable insights, tailor experiences effectively, and evolve their platforms based on genuine user interactions and feedback.

8.0 Deployment Diagram Explanation: Mobile Network Quality of Experience (QoE) App

8.1. Introduction to Deployment Diagrams

A **deployment diagram** is a type of UML (Unified Modeling Language) diagram that visualizes **where** software components run and **how** they communicate in a system. It helps answer questions such as:

- **Where** are the components deployed (e.g., devices, servers)?
- **What** components are running at each location?
- **How** do these components interact or exchange data?

In this project, the deployment diagram maps the architecture of a **Mobile Network Quality of Experience (QoE) Data Collection App**, showing the physical or logical nodes involved and the software components deployed on each.

8.2. High-Level Architecture

The system architecture consists of four major **nodes** or deployment environments:

1. **Mobile Device (User's Smartphone)**
2. **Cloud Backend Server**
3. **Mobile Network Operator (MNO) System**
4. **Admin Workstation**

Each node contains specific components that work together to collect user experience data, analyze it, and support network optimization and reward mechanisms.

8.3. Node-by-Node Breakdown

8.3.1 Mobile Device (User's Smartphone)

This is the primary point of interaction for the user. It runs the mobile app and includes the following components:

- **App UI:**
The graphical interface that users interact with. Includes buttons, forms, graphs, and visual indicators for feedback and rewards.
- **Feedback Collector:**
Enables users to submit ratings or comments on their network experience.
- **Network Logger:**
Continuously measures Quality of Service (QoS) parameters such as signal strength, upload/download speed, latency, and jitter—often in the background.
- **Offline Storage:**
Temporarily stores user feedback and network data locally when the device is offline, syncing with the cloud once connectivity is restored.
- **Location Tracker (Optional):**
Captures GPS or network-based location data to help correlate network issues with geographic areas.
- **Reward Trigger:**
Notifies the backend when a user qualifies for a reward (e.g., airtime or data), based on participation or feedback value.

Communication:

Connects securely to the Cloud Backend Server via encrypted protocols (e.g., HTTPS/TLS).

8.3.2 Cloud Backend Server

Acts as the central processing and storage unit for the application.

- **Secure Storage:**
Maintains user-submitted data (feedback, network logs, locations) in encrypted databases with access controls.
- **Analysis Tool:**
Processes incoming data to detect patterns, identify network problems, and assess service quality trends over time.
- **Reward System:**
Coordinates with the Mobile Network Operator's system to distribute rewards to users, such as mobile data or airtime.
- **API Gateway:**
Manages and secures the communication interface between the mobile app and the backend, ensuring proper request routing and load balancing.
- **Privacy Manager:**
Implements privacy policies, anonymizes sensitive user data, and ensures compliance with data protection regulations (e.g., GDPR, local laws).

8.3.3 Mobile Network Operator (MNO) System

This node represents the systems managed by telecom providers. They receive feedback from the backend and respond accordingly.

- **Data Interface:**
Receives real-time or batch-transferred network quality reports to assess infrastructure performance.
- **Optimization Tools:**
Analyzes aggregated feedback to identify poor-performing cells, dropped call zones, or high-latency areas—feeding into automated or manual network optimizations.
- **Reward API:**
Enables automated delivery of incentives like mobile data or airtime in response to reward triggers from the backend.

8.3.4 Admin Workstation

Used by system administrators and analysts to oversee operations and extract insights.

- **Admin Dashboard:**
A web-based interface for monitoring user activity, system health, feedback trends, and backend logs.
- **Report Generator:**
Creates visual and statistical reports for internal audits, performance reviews, or external reporting to regulators or stakeholders.

8.4. Communication and Security

All communication between nodes is performed over **secure, encrypted channels**, typically using HTTPS or VPN tunneling. Data can be transmitted:

- **In real-time** for critical feedback and reward processing.
- **In batches** when connectivity is limited or delayed.

Security measures include:

- End-to-end encryption
- Access control and authentication (OAuth, tokens)
- Data anonymization
- Role-based user access

8.5. Real-World Analogy

To simplify:

- **Mobile Device = Mail Carrier**
Collects and submits reports (feedback and data).
- **Cloud Backend Server = Post Office**
Sorts, analyzes, and forwards the information.
- **MNO System = City Maintenance**
Uses the reports to improve infrastructure and deliver service tokens.

- **Admin Workstation = Control Room**
Monitors everything and ensures smooth operation.

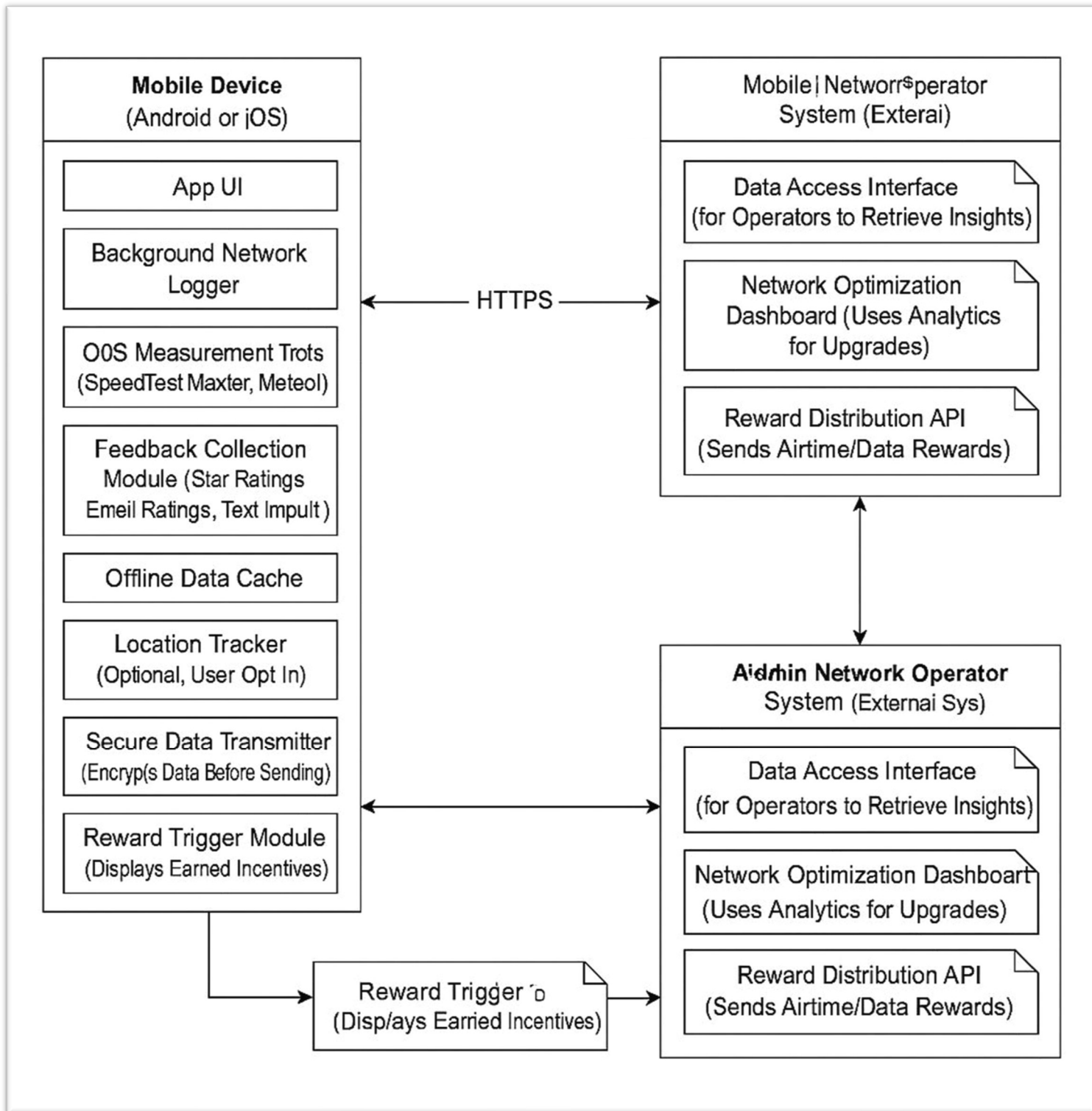


Fig 8.0 depicts the deployment diagram

8.6. Conclusion

The deployment diagram for the Mobile Network QoE App provides a clear view of how the system's components are distributed across different nodes and how they interact to fulfill the application's objectives. Each node—whether it's the user's mobile device, the backend server, the mobile network operator's system, or the admin workstation—plays a distinct but interconnected role in collecting, processing, and utilizing user experience data.

By organizing components in this modular and secure architecture, the system ensures reliable performance, data privacy, and actionable insights. This structure not only enhances the quality of user feedback collection but also enables mobile network operators to respond proactively to service issues and improve overall user satisfaction.

The deployment diagram serves as a foundational tool for understanding system behavior, supporting both development and operational decision-making throughout the lifecycle of the project.

9. Overall Conclusion of System Design and Modeling

The System Design and Modeling phase for the Mobile QoE Data Collection Application is now complete. Through the detailed development of Context, Data Flow, Use Case, Sequence, Class diagram, and Deployment diagrams, a clear architectural blueprint has been established.

This design effectively translates the validated requirements into a tangible structure, outlining how the system will collect both subjective user feedback and objective network performance metrics (including **Location, Timestamps, Network Type, Signal Strength, Upload/Download Speeds, Latency, Jitter, Packet Loss, and Bandwidth**). It defines user interactions (including standard registration and login), data processing flows, and the physical deployment topology, ensuring the core project objective of providing actionable QoE insights to MNOs in Cameroon is met.

This comprehensive set of design models provides a solid foundation for the development phase.